

DOCKER MASTERY

Level 2

Building Images Like an Engineer

A complete reproduction of every concept, question, and discussion from your Level 2 learning journey. Topics 1 to 3 covered in full depth — from writing your first Dockerfile, through layer caching internals, to mastering CMD vs ENTRYPOINT.

Every question you asked is preserved. Every explanation is reproduced. Designed so you never need to look anywhere else.

01 Writing Dockerfiles
Every instruction · build context · real-world template

02 Image Layers & Build Caching
How caching works · cascading invalidation · the golden rule · where layers live

03 CMD vs ENTRYPOINT
The most-confused topic · the override rule · when to use which

Before You Start

This document is built from the actual conversation you had during your Level 2 journey. It captures every question you asked, every concept that was explained, every visualization, and every hands-on exercise. Think of it as a complete, self-contained reference book.

How Level 2 Builds on Level 1

In Level 1 you learned what Docker *is* — architecture, images, containers, the CLI, and registries. You learned to **use** Docker. Level 2 is where you stop being a Docker user and start being a Docker engineer. You don't just run images anymore. You build them. You decide how layers are structured. You design Dockerfiles that build in seconds instead of minutes. You write images that companies actually want to run in production.

The Mindset Shift

Most beginners write Dockerfiles that work. Few write Dockerfiles that work *well*. By the end of Level 2 you will be in the second group — and crucially, you will be able to explain **why** your Dockerfile is the way it is. That is the difference between a junior engineer and a senior one.

HOW TO USE THIS DOCUMENT

Read top to bottom. Don't skip the hands-on exercises — they are not optional. The questions you originally asked are preserved as boxed "Your Question" sections so you can see what triggered each deep-dive. Treat this as your textbook. The concepts here are the foundation for everything else in DevOps.

TOPIC 01

Writing Dockerfiles

every instruction explained · build context · production template

What a Dockerfile Actually Is

A Dockerfile is just a plain text file with a list of instructions. Docker reads it top to bottom and builds an image one layer at a time. That's it. The "magic" is just understanding what each instruction does.

Each instruction in your Dockerfile becomes one layer in the resulting image. Five lines in your Dockerfile = five layers in your image. This is the central fact you build everything else on top of.

The Anatomy of a Dockerfile

Below is a real production-quality Dockerfile for a Python web app. On the left is the file. On the right is the resulting image stack. Notice how each instruction creates a corresponding layer.

Dockerfile (text)	→	Image layer
FROM python:3.11-slim	<i>produces</i>	Layer 1 — Base OS + Python
WORKDIR /app	<i>produces</i>	Layer 2 — /app folder created
COPY requirements.txt .	<i>produces</i>	Layer 3 — requirements.txt added
RUN pip install -r requirements.txt	<i>produces</i>	Layer 4 — Dependencies installed
COPY . .	<i>produces</i>	Layer 5 — Your code added
ENV PORT=5000	<i>produces</i>	metadata only (no real layer)
EXPOSE 5000	<i>produces</i>	metadata only (no real layer)
CMD ["python", "app.py"]	<i>produces</i>	metadata only (no real layer)

each Dockerfile line maps to one layer in the image

METADATA VS REAL LAYERS

Note: ENV, EXPOSE, CMD, ENTRYPOINT, USER, WORKDIR are *metadata only* — they don't run commands or change files. They modify the image's config JSON. That's why they show up in `docker history` with size 0B.

Every Instruction Explained

FROM — the base image

Every Dockerfile must start with FROM. It tells Docker what existing image to build on top of. The choice matters more than people realize — it directly affects image size, security, and package compatibility.

```
FROM python:3.11-slim
FROM node:20-alpine
FROM ubuntu:22.04
FROM nginx:1.25
```

Variant comparison:

Tag	Size	When to use
python:3.11	~900 MB	rarely — too big for production
python:3.11-slim	~45 MB	the sweet spot — recommended default
python:3.11-alpine	~17 MB	smallest, but breaks C-extension packages

Stick with `slim` until you have a specific reason to change. Alpine uses `musl` instead of `glibc`, which causes subtle bugs with Python packages like `numpy`, `pandas`, and `psycopg2`.

WORKDIR — set the working directory

Sets the "current directory" for every instruction that follows — and for the running container. If the directory doesn't exist, Docker creates it. Think of it as `cd /app` that *persists* across instructions.

```
WORKDIR /app
```

```
# Now every following COPY, RUN, CMD operates from /app
```

CRITICAL — WORKDIR VS CD

Always use `WORKDIR /app`, never `RUN cd /app`. `cd` only affects that single `RUN` — each new `RUN` gets a fresh shell that forgets the previous one. `WORKDIR` persists for the whole build.

COPY — copy files into the image

Takes files from your build context (the directory you ran `docker build` from) and puts them inside the image. The order of `COPY` commands is your most important performance lever — we'll cover this in Topic 2.

```
COPY requirements.txt .          # one file, into WORKDIR
COPY . .                        # everything in build context
COPY src/ /app/src/             # specific folder
COPY --chown=appuser:appuser . . # set ownership while copying
```

ADD — like COPY but with extra powers (avoid)

ADD can do everything COPY does, plus download from URLs and auto-extract tar archives. **Don't use it.** The implicit behavior is confusing and a security risk. Use COPY for files, and RUN `curl` if you need to download something — that way the action is explicit. Even Docker's own best-practices docs recommend COPY over ADD.

RUN — execute commands during the build

Runs a command and commits the result as a new layer. This is where you install packages, create users, set up directories — anything that needs to happen *while building* the image.

Critical pattern — chain commands with `&&`:

```
# BAD — three layers, each adds permanent weight
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOOD — one layer, smaller image
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*
```

Always clean up package caches in the same RUN that installed them. Otherwise the cache stays in a layer below and adds permanent weight to your image.

ENV — set environment variables

Environment variables baked into the image and available to the running container. They can also be referenced in later Dockerfile instructions.

```
ENV PORT=5000
ENV NODE_ENV=production
ENV PYTHONUNBUFFERED=1

# can be referenced later:
ENV APP_HOME=/app
WORKDIR $APP_HOME
```

SECURITY

Never put secrets in ENV — passwords, API keys, tokens. They'll be permanently visible in the image with `docker history`. Pass secrets at runtime with `docker run -e` or use a secrets manager.

EXPOSE — document which ports the app uses

Here's the trap: **EXPOSE doesn't actually publish or open any ports**. It's purely documentation. To actually map a port you need `docker run -p 8080:5000`. Think of EXPOSE as saying "this image listens on these ports" — useful info for whoever runs the container.

```
EXPOSE 5000
EXPOSE 80 443
```

USER — run as a non-root user (security)

By default, containers run as **root**. In production this is a security risk — if your app is compromised and someone breaks out of the container, they have root on the underlying system. Always create a non-root user and switch to it before the CMD. This is one of the easiest security wins and almost no beginner does it.

```
RUN useradd -m -u 1000 appuser && \
    chown -R appuser:appuser /app
USER appuser
```

CMD — the default command when the container runs

What runs when the container starts. There can only be one CMD per Dockerfile — if multiple, only the last takes effect. Always use the JSON array (exec) form, not bare strings.

```
CMD ["python", "app.py"] # ✓ exec form (preferred)
CMD python app.py       # ✗ shell form (avoid)
```

We will cover CMD vs ENTRYPOINT in full in Topic 3.

A Complete Production-Quality Dockerfile

Putting every instruction together — this is what a real, professional Python web-app Dockerfile looks like. Read it line by line. Every instruction here is intentional.

```
# Use a small, official base image with a pinned version
FROM python:3.11-slim

# Set working directory inside the image
WORKDIR /app

# Install OS dependencies in one layer, clean up after
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc && \
    rm -rf /var/lib/apt/lists/*

# Copy and install Python deps FIRST (cache-friendly)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Set environment variables
ENV PYTHONUNBUFFERED=1 \
    PORT=5000

# Create a non-root user and switch to it
RUN useradd -m -u 1000 appuser && \
    chown -R appuser:appuser /app
USER appuser

# Document the port
EXPOSE 5000

# Default command
CMD ["python", "app.py"]
```

Notice how `requirements.txt` is copied and installed *before* the rest of the code — that is deliberate caching. Notice the non-root user. Notice the `--no-cache-dir` flag on `pip` to keep the image small. Every choice is intentional.

The Dockerfile You Wrote — Detailed Review

During this journey, you wrote your own real Dockerfile for a Python project. Here it is — annotated with feedback:

```
# Step 1: Start with a Python base image
FROM python:3.11-slim

# Step 2: Set the working directory inside the container
WORKDIR /app

# Step 3: Copy requirements first (Docker caches this layer)
COPY requirements.txt .

# Step 4: Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Step 5: Copy the rest of the code
COPY src/ src/

# Step 6: Expose the port the app runs on
EXPOSE 5000

# Step 7: Command to run the app
CMD ["gunicorn", "--workers=2", "--bind=0.0.0.0:5000", "src.app:app"]
```

What you got right:

Instruction order is correct. `requirements.txt` is copied before `src/`, so the dependency layer gets cached.

Used `--no-cache-dir` on `pip`, keeping the image lean.

Picked the right base image. `python:3.11-slim` is the sweet spot.

Used `gunicorn` instead of the `Flask dev server` — this shows production thinking.

Bound to `0.0.0.0`, not `127.0.0.1`. Most beginners get this wrong — they bind to `localhost` inside the container, then wonder why their app is unreachable from the host.

What's missing — fix these before production:

No non-root USER. Right now your container runs as `root`. Add a user before `CMD` — it's two extra lines and the easiest security upgrade you can make:

```
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser
```

Add this just before the `CMD`. Rebuild. Run `docker run --rm -it myapp:v2 whoami` — should now print `appuser` instead of `root`.

Your Questions — Topic 1

YOUR QUESTION #1

Why do we use this python image in the first instruction? What is this, why do we need here?

The core idea

A Docker image is just a packaged Linux system + your app. You don't want to build it from zero. When your Python app runs, it needs *a lot* of stuff to actually work:

- A Linux operating system (the kernel handles file I/O, networking, processes)
- Basic OS utilities — `ls`, `cat`, `bash`, filesystems, libraries like `libc`
- Python itself (the `python` binary)
- `pip` (Python's package manager)
- All of Python's standard library

If you started from a totally empty image, you'd have to install all of this yourself — download Linux, configure it, compile Python from source, set up `pip`, configure SSL certificates so `pip` can talk to PyPI. You'd spend two days before you got to your code.

FROM `python:3.11-slim` says: **"give me a pre-built image that already has Linux + Python 3.11 + `pip` configured and ready."** It's a starting point, not a final product. Someone at the Python project has already done all the setup work, tested it, secured it, and published it on Docker Hub. You stand on their shoulders.

Decoding the name

```
python:3.11-slim
■      ■      ■
■      ■      ■■■ variant: 'slim' = stripped Debian (~45 MB)
■      ■                  no variant = full Debian (~900 MB)
■      ■                  'alpine' = Alpine Linux (~5 MB)
■      ■■■■■■■■■ Python version: 3.11
■■■■■■■■■■■■■■■■ repository name on Docker Hub
```

Prove it to yourself:

```
# Run the python:3.11-slim image directly with no Dockerfile
docker run --rm -it python:3.11-slim bash

# Now you're inside it — this is what your Dockerfile starts from
cat /etc/os-release          # Debian Linux
python --version             # Python 3.11.x
pip --version                # pip already installed
ls /usr/local/lib/python3.11/ # standard library
exit
```

YOUR QUESTION #2

WORKDIR /app — what does this command do? Does it create any folder inside docker image? I don't understand.

Yes — it does both

WORKDIR creates the folder if it doesn't exist, **AND** switches into it. Two actions in one:

1. If /app doesn't exist inside the image, create it (like `mkdir -p /app`).
2. Set /app as the "current directory" for every instruction that follows — and for the running container itself.

Think of it exactly like opening a terminal and typing `cd /app`, except this `cd` *persists* across every following Dockerfile instruction.

Why do we even need a working directory?

Because dumping your app at / would mix your code with the OS files. The base image already has /bin, /usr, /etc, /var, /lib — all the OS directories. Your code would get tangled in there. By creating /app you give your application its own clean home, separate from the OS.

The convention /app is just a community standard. You could use /myapp, /code, /srv/app — Docker doesn't care. But /app makes your Dockerfile readable to any developer.

Without WORKDIR vs with WORKDIR

```
# WITHOUT WORKDIR — messy
FROM python:3.11-slim
COPY requirements.txt .      # ← copies to /
RUN pip install -r requirements.txt
COPY src/ src/              # ← creates /src
CMD ["python", "src/app.py"] # ← runs from /

# WITH WORKDIR — clean
FROM python:3.11-slim
WORKDIR /app                # ← creates /app, cd into it
COPY requirements.txt .      # ← copies to /app/requirements.txt
RUN pip install -r requirements.txt
COPY src/ src/              # ← creates /app/src
CMD ["python", "src/app.py"] # ← runs from /app
```

Everything for your app lives under /app. Clean separation between your stuff and the OS.

WORKDIR VS RUN CD

`RUN cd /app` only affects that single RUN, and each new RUN gets a fresh shell that doesn't remember the previous one. **WORKDIR is permanent. Always use WORKDIR.**

YOUR QUESTION #3

COPY requirements.txt . — where are they copying all requirements to?

Short answer: it copies the file *from your laptop* into `/app/requirements.txt` inside the image.

The format

```
COPY <source> <destination>
COPY requirements.txt .
  ↑           ↑
  █           █ DESTINATION (inside the image)
  █           █ SOURCE (on your laptop, in build context)
```

That single dot `.` at the end is the destination, and it confuses everyone the first time. **The destination dot means "the current WORKDIR"**, which you set to `/app`. So this command is equivalent to:

```
COPY requirements.txt /app/requirements.txt
```

Both forms are valid. The dot version is shorter and idiomatic.

Visualizing the two sides

YOUR LAPTOP	INSIDE THE IMAGE
████████████████████	████████████████████
flask/	/
██ Dockerfile	██ bin/
██ requirements.txt --COPY-->	██ etc/
██ src/	██ usr/
█ ██ app.py	██ app/ ← WORKDIR
	██ requirements.txt ← here

Other common COPY forms

<code>COPY requirements.txt .</code>	<code># → /app/requirements.txt</code>
<code>COPY requirements.txt ./</code>	<code># → same thing</code>
<code>COPY requirements.txt /app/</code>	<code># → /app/requirements.txt (explicit)</code>
<code>COPY requirements.txt /tmp/req.txt</code>	<code># → renamed and placed elsewhere</code>
<code>COPY src/ src/</code>	<code># → /app/src/</code>
<code>COPY . .</code>	<code># → copies WHOLE build context to /app/</code>

WHY NOT JUST COPY EVERYTHING AT ONCE?

Because of layer caching. If you copy everything at once, every code change invalidates the dependencies layer too — and pip has to reinstall from scratch. By copying `requirements.txt` separately first, the `pip-install` layer stays cached unless you actually change your dependencies. This single trick is what makes Python Docker builds fast. We dive deep into this in Topic 2.

YOUR QUESTION #4

What happens after I run `docker build`? Decode step by step — what is the backend process?

You wanted to understand exactly what happens inside Docker after you press Enter, not just the outputs.

The 9-step pipeline

When you type `docker build -t myapp:v1 .` and press Enter, here is what actually happens behind the scenes:

- 1 Docker CLI parses your command**
Reads the flags · finds your Dockerfile · identifies '.' as build context
- 2 CLI packs your folder into a tarball**
Honors `.dockerignore` · zips up your project · ships it to the daemon over `/var/run/docker.sock`
- 3 Daemon receives and hands off to BuildKit**
Extracts tarball into `/var/lib/docker/tmp` · BuildKit takes over from here
- 4 BuildKit parses the Dockerfile into a plan**
Validates syntax · builds a graph of steps · figures out which can run in parallel
- 5 For each step: check the cache first**
Match? Reuse the saved layer (CACHED ✓). No match? Execute the step.
- 6 Cache miss? Run the instruction in a temporary container**
Spin up tiny container from previous layer · run command · capture diff · save as new layer · throw container away
- 7 Repeat for EVERY instruction**
FROM → WORKDIR → COPY → RUN → COPY → CMD ... each one does the cache-check + run dance
- 8 All steps done — assemble the final image**
Stack layers · write manifest (JSON listing layers) · write config (env vars, CMD) · compute SHA256 = image ID
- 9 Save image and attach the tag `myapp:v1`**
Tag is a friendly pointer to the SHA · prompt returns to you · build done

CRITICAL — BUILD ≠ RUN

docker build does NOT run your container. Build = baking the cake. Run = eating the cake. After build finishes: an image exists on disk with tag `myapp:v1`, but nothing is running, no container exists, your app is NOT executing. To actually start your app, you need `docker run`.

Decoding YOUR Real Build Output

When you ran your build, you got this output. Here is every line decoded:

```
$ docker build -t myapp:v1 .

[+] Building 0.7s (11/11) FINISHED          docker:default
                                         ↑ build engine

=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 553B                    ← Dockerfile size

=> [internal] load metadata for python:3.11-slim        0.6s
=> [auth] library/python:pull token                    ← auth with Hub

=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                          ← .dockerignore size

=> [1/5] FROM python:3.11-slim@sha256:6d8537...         0.0s

=> [internal] load build context                        0.0s
=> => transferring context: 390B                        ← your project size

=> CACHED [2/5] WORKDIR /app                            0.0s
=> CACHED [3/5] COPY requirements.txt .                  0.0s
=> CACHED [4/5] RUN pip install ...                     0.0s
=> CACHED [5/5] COPY src/ src/                          0.0s

=> exporting to image                                  0.0s
=> => writing image sha256:ab8e5ef82ba36a32...         0.0s
=> => naming to docker.io/library/myapp:v1
```

Key takeaways from this output

0.7s total · (11/11) FINISHED — blazing fast because almost everything was cached. The (11/11) is Docker's internal step count, not the same as Dockerfile lines.

transferring context: 390B — your project is tiny. If this ever shows MB or GB, you're shipping unnecessary files (likely `node_modules` or `.git`).

transferring context: 2B next to `.dockerignore` — your `.dockerignore` is essentially empty. Topic 5 will fix this.

Four CACHED steps — your layer ordering is correct. The expensive `pip install` didn't rerun because `requirements.txt` didn't change.

writing image sha256:ab8e5ef82ba36a32... — your image's permanent fingerprint. The tag `myapp:v1` is just a pointer to this hash.

YOUR QUESTION #5

How do I know what is happening after I run my Dockerfile? How do I verify and inspect what got built?

Six layers of visibility — from build-time to runtime — that take you from "I hope it works" to "I know exactly what's happening."

1. Watch the build output

Each [N/5] line is one of your instructions becoming a layer. The time on the right tells you which steps are slow. CACHED = layer reused. Errors stop the build right there with a red message — read the line above it.

2. Inspect what got built — without running it

```
# How big is my image?
docker images myapp:v1

# What instructions created what layers? (BEST debugging tool)
docker history myapp:v1

# Full configuration — env vars, exposed ports, default user, CMD
docker inspect myapp:v1
```

`docker history` shows every layer with its size, so you can immediately see what bloated your image. If you see a 200 MB layer next to `RUN apt-get install`, that's where to optimize.

3. Peek inside the image — without starting your app

This is the trick most beginners don't know — override the CMD and drop into a shell to explore the image's filesystem before your app runs:

```
docker run --rm -it myapp:v1 sh

# Now you're inside the image:
ls /app                # is my code there?
ls /app/src            # are my files where I expect?
cat /app/requirements.txt # what got copied?
which gunicorn         # did pip install it correctly?
python -c "import flask" # do imports work?
env                   # what env vars are set?
whoami                 # what user am I? (probably root)
exit
```

4. Read the logs

```
docker logs -f myapp

# you should see:
# [INFO] Starting gunicorn 21.2.0
# [INFO] Listening at: http://0.0.0.0:5000
# [INFO] Booting worker with pid: 7
# [INFO] Booting worker with pid: 8
```

5. Check what it's actually doing

```
docker stats myapp      # live CPU/memory/network
docker top myapp        # processes inside (gunicorn master + 2 workers)
docker port myapp       # port mappings
curl http://localhost:8080 # hit it from outside
```

6. Get inside the running container to debug

```
docker exec -it myapp sh

ls /app          # confirm code is there
ps aux           # see what's running
curl localhost:5000 # does the app respond from inside?
exit
```

If `curl localhost:5000` works inside but not from your host, the problem is port mapping, not your app. This is a powerful diagnostic split.

WHEN SOMETHING GOES WRONG — DIAGNOSTIC FLOW

(1) Did it even start? `docker ps -a | grep myapp` — if STATUS shows Exited (1) or Exited (137), it crashed. **(2) Why?** `docker logs myapp` — the last lines before exit tell you everything. **(3) Common errors:** `ModuleNotFoundError` = missing dep. `address already in use` = port conflict. `permission denied` = file permissions or USER issue.

YOUR QUESTION #6

You said Docker is built on top of Linux. Explain that. I did not understand this.

The big picture in one sentence

Docker did not invent any new technology. It packaged together a bunch of Linux features that already existed since 2008 and gave them a great developer interface. Before Docker existed, every single thing Docker does was already possible on Linux — but it was painful, command-line-heavy, and only sysadmins did it. Docker's genius was making it easy.

The four Linux primitives Docker uses

Namespaces — "isolating what a process can SEE"

A namespace is a way for the Linux kernel to give a process its own private view of the system. There are six main types, and Docker uses all of them:

- **PID namespace** — your process thinks it's the only process on the system. Inside a container, `ps aux` shows only the container's processes.
- **Network namespace** — your process gets its own network stack: own IP, own routing table, own firewall rules.
- **Mount namespace** — your process gets its own filesystem view. It thinks `/` is the container's filesystem.
- **UTS namespace** — own hostname, separate from host.
- **IPC namespace** — cannot communicate with other containers via shared memory.
- **User namespace** — can be "root" inside the container while being a regular user on the host.

This is what creates the illusion of a separate machine. The container is just a regular Linux process, but the kernel is lying to it about what it can see.

cgroups — "limiting what a process can USE"

While namespaces control what a process can see, cgroups control what it can *consume*. They let the kernel say:

- "This process can use at most 1 CPU core."
- "This process can use at most 512 MB of RAM."
- "This process can do at most 100 MB/s of disk I/O."

When you run `docker run --memory=512m --cpus=1 myapp`, Docker translates that into kernel cgroup configuration. The kernel actually enforces the limit. Docker just sets the rules. **cgroups existed in Linux since 2008, originally developed by Google** — long before Docker (2013).

OverlayFS — "stacking layers as one filesystem"

OverlayFS is a Linux filesystem driver that lets you stack multiple read-only directories on top of each other and present them as a single unified filesystem. With one writable directory on top.

This is exactly how Docker images work. Each layer in your image (the OS files, Python install, your dependencies, your code) is a read-only directory. OverlayFS stacks them. When the container runs, a writable layer goes on top.

When you write a file inside the container, OverlayFS uses **copy-on-write** — it copies the original file to the writable top layer and modifies the copy, leaving the underlying read-only layer untouched. (Detailed explanation in the next question.)

Capabilities + seccomp — "restricting dangerous syscalls"

Linux processes can ask the kernel to do things via system calls — open files, mount filesystems, change the system clock. Some are dangerous. **Capabilities** let the kernel grant or deny specific permissions. By default, Docker drops most dangerous capabilities. **seccomp** filters which syscalls a process can make. Docker has a default seccomp profile that blocks ~60 dangerous syscalls.

Why this matters — three big consequences

1. Docker only runs natively on Linux.

If Docker uses Linux kernel features, then on Windows or Mac there is no Linux kernel. **Docker Desktop secretly runs a tiny Linux virtual machine in the background.** All your "Docker containers" on Mac/Windows are actually running inside this hidden Linux VM. On macOS it's called LinuxKit. On Windows it uses WSL2. This is why Docker on Mac/Windows is slightly slower than on Linux — there's a VM layer in between.

2. Containers share the host's kernel.

A virtual machine boots a complete OS, including its own kernel. Heavy. Slow. Isolated. A container is just a process running on the host's kernel with the kernel lying to it. Lightweight. Fast. Less isolated than a VM, because if the kernel has a security hole, every container could be affected.

3. Kubernetes does not need Docker.

Kubernetes orchestrates containers. But containers are a Linux kernel feature, not a Docker feature. **Kubernetes dropped Docker as a runtime in 2022** and moved straight to containerd. Same containers. Same kernel. Just no Docker layer.

Try it on your machine — proof Docker is just Linux

```
# 1. List the namespaces of your current shell
ls -la /proc/self/ns
# you'll see: pid, net, mnt, uts, ipc, user – your namespace identity

# 2. Check the cgroups your shell is in
cat /proc/self/cgroup

# 3. See OverlayFS support
cat /proc/filesystems | grep overlay

# 4. Run a Docker container, look at ITS namespaces
docker run -d --name probe nginx
docker inspect --format '{{.State.Pid}}' probe # get PID
sudo ls -la /proc/<that_pid>/ns
# different namespace IDs from your shell – proof of separate view
```

YOUR QUESTION #7

How does copy-on-write keep the read-only layer untouched? Explain how it works.

The library analogy

Imagine a public library with a shelf of books. Anyone can read these books, but the rule is: **no writing on them**. You walk in with an empty notebook and want to "edit" the recipe in chapter 3. What do you do?

1. You photocopy chapter 3 of the cookbook.
2. You take the photocopy to your desk and edit it freely.
3. The original book on the shelf stays untouched.
4. When YOU read your "edited cookbook", you see the photocopy with your changes.
5. When anyone else reads the cookbook, they see the original.

That's copy-on-write. The original is sacred. Modifications happen on a personal copy that lives on top.

The four stages — what really happens

Stage 1 — Fresh container starts

Image layers (OS, Python, your code) are mounted as the **lowerdirs** — read-only. An empty directory is created as the **upperdir** — writable, specific to this container. A "merged" view is presented — when the container looks at its filesystem, it sees the upper layer + lower layers combined into one seamless filesystem.

Stage 2 — Container READS a file

Suppose your container does `cat /app/app.py`. OverlayFS searches **top-down** — checks **upperdir** first (empty, skip), then layer 3 (your code layer) — found! Reads directly from layer 3. **No copying.** The file is read in place from the read-only layer.

This is why containers can start in milliseconds even from a 500MB image — there's no "copy the image to the container's filesystem" step. The container just reads from existing layers.

Stage 3 — Container WRITES to a file (the magic)

Suppose your container does `echo "new" >> /app/app.py`. Watch the multi-step dance:

- A. Find the file. OverlayFS searches top-down, finds it in layer 3 (read-only).
- B. Realize: write attempt on read-only layer. Cannot modify it directly — that would violate the immutability rule.
- C. Copy the file UP to the writable layer. OverlayFS makes a fresh copy of `/app/app.py` and places it in the **upperdir**. **This is the "copy" in copy-on-write.**
- D. Modify the copy. The kernel writes new content to the copy in **upperdir**. The original file in layer 3 is never touched.

E. Done. The container now has its own version sitting in the writable layer.

Stage 4 — Reading the file again after the write

Container does `cat /app/app.py` a second time. OverlayFS searches top-down, finds it in `upperdir`, returns the modified copy. **The kernel never even reaches layer 3.** The writable layer *shadows* the lower layer.

But — and this is the punchline — **another container running from the same image** that hasn't touched `/app/app.py` still sees the *original* version from layer 3. Two containers see different files even though they're "running the same image" because their writable layers are separate.

Three rules to internalize

Rule 1 — Reads search top-down, return first match. Kernel checks writable layer first, then each image layer below. First file found wins.

Rule 2 — Writes always go to the writable top layer. Kernel never writes to lower (image) layers. They are mounted read-only at the kernel level — the kernel literally cannot write there.

Rule 3 — If a write targets a file in a lower layer, that file is copied up first, then modified. This is the copy-on-write event. It only happens the FIRST time the file is modified — subsequent edits just modify the existing copy.

Three huge implications

1. Image layers are physically shared between containers. If you run 10 containers from `nginx:latest`, the OS files exist *once* on disk. All 10 containers point to the same lower layers. Only their writable upper layers differ. This is why containers are so cheap.

2. The writable layer disappears when the container is removed. When you `docker rm` a container, the `upperdir` gets deleted. All changes that container made — files, logs, data — gone. The image layers are still there, untouched. This is why containers are *ephemeral by default* and why you need **volumes** for persistent data.

3. Writes are slow the first time, fast after. The first write to a file from the image layer triggers a copy. If it's a 500MB log file, OverlayFS copies all 500MB before writing one new byte. After that, the file lives entirely in the upper layer and writes are instant.

Prove it on your machine

```
# 1. Run a fresh nginx container
docker run -d --name web1 nginx

# 2. Get inside it
docker exec -it web1 bash

# 3. Look at the default index page (lives in IMAGE layer)
cat /usr/share/nginx/html/index.html
# default 'Welcome to nginx!' page

# 4. Modify it (triggers COPY-ON-WRITE)
echo "<h1>Hello from web1!</h1>" > /usr/share/nginx/html/index.html

# 5. Read it back – see your version
cat /usr/share/nginx/html/index.html

exit

# 6. Start a SECOND container from the SAME image
docker run -d --name web2 nginx
docker exec -it web2 cat /usr/share/nginx/html/index.html
# original 'Welcome to nginx!' page!
# Proof: image layer was never touched. web1 only modified its OWN copy.

# Cleanup
docker stop web1 web2 && docker rm web1 web2
```

THE MOMENT IT CLICKS

Run that experiment. The instant you see web2 showing the original page even though web1 modified the "same file", copy-on-write becomes permanently obvious in your head.

Hands-on Summary for Topic 1

```
# Build the production-quality image
docker build -t myapp:v1 .

# Inspect what got built
docker images myapp:v1
docker history myapp:v1
docker inspect myapp:v1

# Peek inside without running your app
docker run --rm -it myapp:v1 sh
ls /app
exit

# Run for real
docker run -d -p 8080:5000 --name myapp myapp:v1
docker logs myapp
curl http://localhost:8080

# Cleanup
docker stop myapp && docker rm myapp
```

TOPIC 02

Image Layers & Build Caching

the most underrated skill in Docker · where layers live · the domino effect

Part A — Layer Caching, Made Simple

We're going to build this up like Lego blocks. Don't move on until each section feels obvious.

Idea #1 — A Dockerfile is a recipe

Imagine baking a cake. The recipe has steps:

1. Preheat oven
2. Mix flour and sugar
3. Add eggs
4. Pour into pan
5. Bake

A Dockerfile is the same — a list of steps. Each step in your Dockerfile creates **one layer** in your image.

LOCK THIS IN

5 lines in your Dockerfile = 5 layers in your image. Same as 5 steps in the recipe = 5 things you did to make the cake.

Idea #2 — Building takes time. Caching saves time.

Imagine you bake the same cake every day. Step 4 is "bake for 45 minutes." That's slow. What if you could SAVE the cake from yesterday and skip baking today? You could! As long as the recipe didn't change.

That's exactly what Docker does. The first time you build, every step actually runs. The second time, Docker says "wait — I already did this, here's the saved result."

First build vs Second build (visualized)

Step	First build (slow)	Second build (fast!)
FROM python:3.11-slim	3 sec	CACHED ✓
WORKDIR /app	0 sec	CACHED ✓
COPY requirements.txt	0 sec	CACHED ✓

RUN pip install	25 sec	CACHED ✓
COPY . .	1 sec	CACHED ✓
TOTAL	29 seconds ■	0.7 seconds ■

same Dockerfile, same project — Docker reused the saved layers

LOCK THIS IN

Caching = Docker remembers what it did last time and reuses it. That's all caching means.

Idea #3 — How does Docker know it can reuse a layer?

Docker has a simple rule. For each step, it asks itself one question:

THE QUESTION

"Has anything about this step changed since last time?"

NO → reuse the saved layer (CACHED ✓)

YES → run the step again (rebuild)

But what counts as "changed"? Three things:

1 ■■■ **Did the LINE itself change?** e.g. you edited "RUN pip install" to "RUN npm install"

2 ■■■ **For COPY: did the FILE CONTENT change?** e.g. you added a new line in requirements.txt

3 ■■■ **Did the layer ABOVE this one change?** If yes, this one also has to rebuild (the domino effect)

LOCK THIS IN

Question 1 — did you change the Dockerfile text on this line? → rebuild. Question 2 — for COPY lines, did you change the file? → rebuild. Question 3 — did anything ABOVE this layer change? → rebuild. **If all three answers are NO → reuse the cached layer.**

Idea #4 — The Domino Effect (most important)

Layers are stacked in order. Layer 1 → 2 → 3 → 4 → 5. **If layer 3 has to rebuild, layers 4 and 5 also have to rebuild** — even if you didn't touch them. Why? Because each layer is built on top of the previous one. If the foundation changes, everything above it has to be rebuilt on the new foundation.

Think of it like a tower of dominoes. Push one in the middle, every domino after it falls.

Visualizing the domino effect

Layer	Status
Layer 1 — FROM python:3.11-slim	CACHED ✓
Layer 2 — WORKDIR /app	CACHED ✓
Layer 3 — COPY requirements.txt ■	CHANGED!
Layer 4 — RUN pip install	REBUILT (cascaded)
Layer 5 — COPY . .	REBUILT (cascaded)

change ONE thing → everything BELOW it rebuilds (above stays cached)

LOCK THIS IN

Change one layer → all the layers BELOW it must rebuild. The layers ABOVE stay cached.

Idea #5 — The Golden Rule

THE ONE RULE

Put things that change **RARELY** at the **TOP**.
Put things that change **OFTEN** at the **BOTTOM**.

Why? Because the domino effect only knocks over things *below* the change. So if "things that change a lot" are at the bottom, they only knock over a few cheap dominoes. If they're at the top, they knock over EVERYTHING.

Bad order vs Good order — side by side

■ BAD ORDER	✓ GOOD ORDER
FROM python:3.11-slim	FROM python:3.11-slim
COPY . . ■ changes	COPY requirements.txt .
RUN pip install (slow)	RUN pip install (slow)
CMD ["python", "app.py"]	COPY . . ■ changes
	CMD ["python", "app.py"]

Bad order: code is at the TOP, code changes constantly, so pip install (below) keeps getting wiped out. (*Edit one source file → 25 seconds wasted reinstalling pip.*)

Good order: code is at the BOTTOM, pip install is ABOVE the code, so code changes don't touch pip install. (*Edit one source file → 0.5 seconds total.*)

THE SINGLE MOST IMPORTANT OPTIMIZATION

Source code goes at the BOTTOM. Why? Because you change source code 100 times a day. Dependencies you change once a week. The frequent thing must be at the bottom so it doesn't keep wiping out the slow thing. **This single reordering converts a 60-second rebuild into a 1-second rebuild.**

Part B — Going Deeper

Now that you have the intuition, here's exactly what's happening underneath.

The cache key formula

Docker computes a fingerprint (cache key) for every instruction. If the fingerprint matches a previously built layer, that layer is reused.

CACHE KEY FORMULA

`cache_key = SHA256( parent_layer_id + instruction_text + file_content* )`

* file_content only matters for COPY and ADD — for RUN, only the instruction text matters.
Match the hash → reuse the layer (CACHED). Else → rebuild from here down.

What invalidates each instruction type

Instruction type	Invalidated by	What does NOT matter
RUN, WORKDIR, ENV, USER	change in instruction text itself	what the command actually fetches
COPY, ADD	SHA-256 of any file being copied changed	file modification time alone
FROM (base image)	tag pointing to new digest on registry	uses local cache by default; use --pull to refresh

The classic anti-pattern (and why it's catastrophic)

Beginners often write this:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

It works! The image runs! But every time you change one line of code, the `COPY . .` layer invalidates. Which cascades to `RUN pip install` — **every dependency reinstalls from scratch on every build**. If you have 30 dependencies, that's a 60-second rebuild for a one-line change. Multiply by hundreds of CI/CD runs per week — you're burning real time and money.

The fix

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt . # only invalidates when deps change
RUN pip install --no-cache-dir -r requirements.txt
COPY . . # invalidates when source changes
CMD ["python", "app.py"]
```

Part C — Where Are Layers Actually Stored?

You asked one of the deepest questions in this topic: *where are these layers data stored, where do they get checked, how does this process happen?* This is the level of curiosity that turns engineers into senior engineers. Most people accept "Docker caches stuff" as magic. You're asking **where** and **how**. Here's the answer.

THE ADDRESS

/var/lib/docker/ on Linux. This is Docker's warehouse. Every image, every layer, every container's writable area, every tag — it all lives here.

The library analogy that explains everything

Think of Docker's storage like a public library:

- **overlay2/** = the **shelves** where the actual books sit
- **image/** = the **card catalog** that tells you which book is on which shelf
- **buildkit/** = a **notebook** where the librarian writes "I already shelved a copy of this exact book — its catalog number is X"

When you ask the library for a book, the librarian doesn't search every shelf. They check the catalog first, get the shelf number, then walk straight to it. That's exactly how Docker handles caching.

The five folders inside /var/lib/docker

Folder	Job
image/	the INDEX — catalog mapping tags → SHA hashes, image manifests, layer references
overlay2/	the WAREHOUSE — actual layer file contents (Linux OS, Python binary, your code)
containers/	running and stopped containers — config + logs
volumes/	named volumes — persistent data that survives container deletion
buildkit/	BuildKit's build cache — source of "CACHED ✓" in your build output

WARNING

Never modify these folders by hand — let Docker manage them. Use `docker images`, `docker history`, `docker inspect` to look inside safely.

overlay2/ — actual file contents

This folder holds the REAL files. The Linux OS, Python binary, your `app.py`, your installed pip packages — all the actual bytes that make up your image — live in here.

```
/var/lib/docker/overlay2/
■■■ 8a7b3c4d5e6f.../
■   ■■■ diff/           ← actual files for one layer
■       ■■■ etc/
■       ■■■ usr/
■       ■■■ ...
■■■ d2f9e1a8b7c6.../
■   ■■■ diff/
■       ■■■ app/
■       ■■■ requirements.txt
■■■ 5f3e2d1c0b9a.../
■   ■■■ diff/
■       ■■■ usr/local/lib/python3.11/site-packages/
■■■ ...hundreds more folders, one per layer
```

Each folder name is a SHA hash. **Each folder is one layer.** Inside each folder is a `diff/` subfolder containing the actual files this layer added or changed. There's no magic — layers are just folders of files on your disk, organized by SHA.

image/ — the catalog

`overlay2/` alone is useless — it's just a pile of folders with cryptic SHA names. You'd never know that those particular folders together make up the `python:3.11-slim` image. That's what `image/` is for. It contains JSON files describing:

- "The image tagged `python:3.11-slim` is `sha256:6d8537...`"
- "That image is made of layers `8a7b3c...`, `d2f9e1...`, `5f3e2d...` in this order"
- "Its config has these env vars, this CMD, this WORKDIR"

When you run `docker images` or `docker inspect`, Docker reads these JSON files. They are the catalog that gives meaning to the raw layer folders in `overlay2/`.

buildkit/ — the cache lookup brain

This is the answer to "where does it check if a layer is cached?" When BuildKit processes your Dockerfile, every time it computes a cache key for a step, it asks the `buildkit/` cache: "have I seen this exact cache key before?"

If yes → "great, I already have a layer for that. Here's its SHA — go grab it from `overlay2/`."
→ **CACHED ✓**. If no → never seen this key. Has to actually run the step and create a new layer.

The lookup happens in a tiny key-value database stored in `buildkit/`. The keys are cache fingerprints. The values are pointers to layers in `overlay2/`.

The complete picture — three folders working together

Cache HIT sequence:

1. Compute the cache key for this Dockerfile instruction.
2. Ask buildkit/: "do you have an entry for this key?"
3. If yes, buildkit/ returns the SHA of the matching layer.
4. image/ is updated to record "this image now uses layer X."
5. overlay2/ is told to mount layer X as part of the new image's filesystem.

No command runs. No download. No work. Just a dictionary lookup followed by reusing existing files on disk. That's why CACHED steps take 0.0 seconds.

Cache MISS sequence:

1. Compute the cache key.
2. Look up in buildkit/ — not found.
3. Spin up a temporary container, actually run the command.
4. Capture the diff (changed files) and write to a new folder under overlay2/.
5. Compute the SHA of the new layer.
6. Record in buildkit/: "key X → layer SHA Y."
7. Update image/ with the new layer.

Now next time you build with the same instruction and unchanged files, the lookup in step 2 will succeed and skip steps 3-7.

See your own cache with your own eyes

```
# 1. Where is Docker's storage?
docker info | grep "Docker Root Dir"
# → /var/lib/docker

# 2. See the warehouse layout
sudo ls /var/lib/docker
# → image/ overlay2/ containers/ buildkit/ volumes/ ...

# 3. Count how many layers are stored on your machine
sudo ls /var/lib/docker/overlay2 | wc -l
# Often surprising – could be 100+ even after a few builds

# 4. See total disk usage by Docker
docker system df

# 5. See more detail
docker system df -v
```

MAC / WINDOWS NOTE

If you're on macOS or Windows running Docker Desktop, you won't find `/var/lib/docker` on your Mac/Windows filesystem. That folder lives **inside a Linux VM** that Docker Desktop secretly runs in the background. From your perspective, the storage is hidden away in a single VM disk file. You can still inspect everything via `docker images`, `docker history`, `docker system df` — those commands all talk to the daemon inside the VM.

Part D — Hands-on Experiment (lock it in)

Reading is fine. Watching it happen on your machine is what makes it stick. Try these 4 builds in order — it takes 5 minutes:

```
# ■■ Test 1: Fresh build, nothing cached ■■
docker build --no-cache -t myapp:v1 .
# ■ Watch: every step actually runs.
# Note how long pip install takes.

# ■■ Test 2: Build again, no changes ■■
docker build -t myapp:v1 .
# ■ Watch: every step says "CACHED ✓".
# Build finishes in ~1 second.
# THIS IS BUILDKIT looking up cache keys in its database.
# Each lookup is a fast dictionary check, not a real execution.

# ■■ Test 3: Edit a source file ■■
echo "# hello" >> src/app.py
docker build -t myapp:v1 .
# ■ Watch: COPY src/ src/ rebuilds.
# Everything ABOVE it stays CACHED.
# The expensive pip install was SAVED. This is the win.

# ■■ Test 4: Edit requirements.txt ■■
echo "" >> requirements.txt
docker build -t myapp:v1 .
# ■ Watch: COPY requirements.txt invalidates →
# pip install reruns from scratch → COPY src/ also reruns.
# This is the domino effect in action.

# ■■ See the BuildKit cache directly ■■
docker buildx du
# → shows you cache entries with their sizes

docker buildx du --verbose
# → see individual cache entries
```

THE MOMENT IT CLICKS

When you watch the "CACHED" word appear and disappear depending on what you change, layer caching becomes obvious forever. Run the experiment.

Part E — Self-Check (5 questions)

You should be able to answer all 5 instantly:

Q1. What is a layer?

→ One step in the Dockerfile that becomes one part of the image.

Q2. When does Docker reuse a saved layer?

→ When the line text is unchanged, files (for COPY) are unchanged, AND the layers above haven't changed.

Q3. What's the domino effect?

→ If one layer rebuilds, every layer below it also has to rebuild.

Q4. What's the golden rule?

→ Put rarely-changing things at the top, frequently-changing things at the bottom.

Q5. Why is COPY . . before pip install bad?

→ Because every time you change a source file, pip install (the slow expensive step) is forced to rerun from scratch.

Summary in one paragraph

TOPIC 2 — THE WHOLE THING

Each line in your Dockerfile becomes a layer. Docker saves layers and reuses them on the next build to save time. A layer is reused only if (a) the line text didn't change, (b) any files being copied didn't change, and (c) the layers above didn't change. If any layer rebuilds, every layer below it also rebuilds — this is the domino effect. So the golden rule is: put things that rarely change (like installing dependencies) at the top, and put things that change often (like your source code) at the bottom. This means editing your code only rebuilds the cheap last layer, while the expensive steps stay cached. Layers are stored in `/var/lib/docker/overlay2/`, indexed by `/var/lib/docker/image/`, and the cache lookup happens in `/var/lib/docker/buildkit/`.

TOPIC 03

CMD vs ENTRYPOINT

the most-confused topic in Docker · made crystal clear in one shot

This is the most-confused topic in Docker. Engineers years into their career still mess this up. We're going to fix that today, in one go, with so many examples and pictures that it becomes obvious. Take a breath. We'll build it up slowly.

Idea #1 — Why Docker needs a default command

When you run a container, Docker has to run *something* inside it. A container is just a process. You can't have a container without a process to run.

So somewhere, somehow, Docker needs to know: **"when this container starts, what should it execute?"**

There are three ways to tell Docker:

1. Put it in the Dockerfile (CMD or ENTRYPOINT)
2. Pass it on the command line (`docker run myimage <command>`)
3. Both

CMD and ENTRYPOINT are the two Dockerfile instructions for #1. They both *seem* to do the same thing — but they behave very differently when you start mixing in command-line arguments.

THE CONFUSION IN ONE SENTENCE

The difference between CMD and ENTRYPOINT only matters when you pass extra arguments to `docker run`.

The simplest mental model

ENTRYPOINT (the program)	CMD (the default arguments)
always runs	a default that gets used
cannot be replaced easily	if you don't override it
echo (the command itself)	"hello world" (what you pass to it)

Final command run inside container = ENTRYPOINT + CMD

LOCK THIS IN

ENTRYPOINT = the program (the command itself).

CMD = the default arguments to that program.

When the container starts, Docker glues them together: ENTRYPOINT first, then CMD. So the actual command becomes [ENTRYPOINT] [CMD].

If that's all you remember, you'll get 80% of cases right.

Idea #2 — The two forms (the syntax trap)

Both CMD and ENTRYPOINT can be written in two different ways. **One is correct. The other is mostly wrong.**

```
# ✓ EXEC FORM (use this – JSON array)
CMD ["python", "app.py"]
ENTRYPOINT ["python", "app.py"]

# ✗ SHELL FORM (avoid – bare string)
CMD python app.py
ENTRYPOINT python app.py
```

The difference looks tiny. The behavior is huge.

What actually runs inside the container

✓ EXEC FORM	✗ SHELL FORM
CMD ["python", "app.py"]	CMD python app.py
runs DIRECTLY:	runs THROUGH a shell:
python app.py	/bin/sh -c "python app.py"
PID 1 = python	PID 1 = sh, then sh starts python

Why shell form is bad — 3 real problems

- 1. Signals don't reach your app.** When you `docker stop`, the signal goes to `/bin/sh` — your Python app doesn't shut down gracefully.
- 2. Extra process wastes a tiny bit of memory.** Your container has TWO processes (`sh` + `python`) instead of one.
- 3. ENTRYPOINT in shell form silently drops docker run arguments.** Huge gotcha — extra args you pass on the command line are completely lost.

LOCK THIS IN

ALWAYS use the exec form (the JSON array `["...", "..."]`). Shell form looks easier but breaks signals (your app won't shut down cleanly when Kubernetes/Docker tries to stop it) and silently swallows command-line arguments. The only exception is when you genuinely need shell features like `env-var` expansion or pipes — even then, prefer `CMD ["sh", "-c", "cmd | other"]` to stay explicit.

Idea #3 — The ONE rule that explains everything

THE OVERRIDE RULE

When you `docker run myimage <extra-args>`:
the extra args **REPLACE CMD** — but they get **APPENDED** to **ENTRYPOINT**.

Said another way:

CMD is overridable. Pass anything to docker run? Your CMD is gone, replaced.

ENTRYPOINT is permanent. Pass anything to docker run? It just gets added on as arguments.

That's it. That's the whole thing. 4 cases prove it:

Case 1 — only CMD, no override

```
Dockerfile: CMD ["python", "app.py"]
Run as:      docker run myimage

→ runs:  python app.py  ✓ (uses the default)
```

Case 2 — only CMD, WITH override

```
Dockerfile: CMD ["python", "app.py"]
Run as:      docker run myimage echo hi

→ runs:  echo hi  ■ (CMD was REPLACED — your app didn't run!)
```

THE SILENT KILLER

Case 2 is what trips beginners. They don't realize CMD silently disappears the moment any args are added to `docker run`. The Python app never starts. The container just echoes "hi" and exits. People stare at this for hours.

Case 3 — only ENTRYPOINT, no override

```
Dockerfile: ENTRYPOINT ["python", "app.py"]
Run as:      docker run myimage

→ runs:  python app.py  ✓ (always runs)
```

Case 4 — only ENTRYPOINT, WITH "override"

```
Dockerfile: ENTRYPOINT ["python", "app.py"]
Run as:      docker run myimage --debug --port=5000

→ runs:  python app.py --debug --port=5000  (args APPENDED!)
```

THE BIG TAKEAWAY

CMD: replaceable default. **ENTRYPOINT:** locked-in command + appended args.

Idea #4 — When you use BOTH together

Now the cool part. When you put ENTRYPOINT and CMD in the same Dockerfile, they combine. **ENTRYPOINT is the program. CMD is the default arguments to that program.**

```
ENTRYPOINT ["python"]
CMD ["app.py"]
```

When you run this:

```
docker run myimage
# → runs: python app.py      (ENTRYPOINT + CMD glued together)

docker run myimage other.py
# → runs: python other.py    (CMD replaced with "other.py", ENTRYPOINT stays)

docker run myimage --version
# → runs: python --version   (CMD replaced, ENTRYPOINT stays)
```

This is the real purpose of having two instructions. ENTRYPOINT locks in the program. CMD provides smart defaults that the user can override.

Real-world example: the official python image

```
# Official python image (simplified)
ENTRYPOINT ["python3"]
CMD ["python3"] # default to interactive REPL
```

So `docker run python:3.11` drops you into the Python REPL. But `docker run python:3.11 myscript.py` runs your script. Same image, different behaviors, controlled by what you append. **That's the elegance.**

Idea #5 — When to use what (decision tree)

Now you have the pieces. Here's how to decide for any given Dockerfile:

Your image is...	Use this pattern	Example
a fixed web app (Flask/Node/Django)	ONLY CMD	<code>CMD ["gunicorn", "--bind=0.0.0.0:5000", "src.app:app"]</code>
a CLI tool (like aws, kubectl)	ENTRYPOINT + CMD	<code>ENTRYPOINT ["aws"] CMD ["--help"]</code>
a strict server that cannot be replaced	ONLY ENTRYPOINT (rare)	<code>ENTRYPOINT ["nginx", "-g", "daemon off;"]</code>

RULE OF THUMB (MEMORIZE THIS)

Building an APP image (your Flask, Node, Django)?

→ Use just `CMD [". . ."]`.

Building a TOOL image (a CLI you'd type subcommands to)?

→ Use `ENTRYPOINT ["tool"] + CMD ["default-args"]`.

99% of beginner Dockerfiles want CMD only — that's the right default.

Idea #6 — How to override either one

Sometimes you want to break the rules and override either CMD or ENTRYPOINT from the command line. Here's the complete toolkit:

```
# Just append args (replaces CMD, leaves ENTRYPOINT alone)
docker run myimage <args>

# Override ENTRYPOINT (rare, but useful for debugging)
docker run --entrypoint <new-cmd> myimage <args>

# COMMON DEBUGGING TRICK: get a shell instead of running the app
docker run --entrypoint sh -it myimage
# → drops you into a shell so you can poke around the image
```

The **--entrypoint flag** is your **debugging superpower**. When your container won't start and you need to look around inside the image, `--entrypoint sh -it` overrides the default and gives you a shell.

Idea #7 — Decode YOUR real Dockerfile

Your real Dockerfile has this:

```
CMD ["unicorn", "--workers=2", "--bind=0.0.0.0:5000", "src.app:app"]
```

Let's decode every aspect:

You used CMD only. ✓ Correct choice for an app image.

You used exec form (JSON array). ✓ Correct — signals will reach unicorn properly.

The args are in the array, not separate. ✓ Each token is its own quoted string. This is correct.

What happens when you run it different ways:

```
# Normal — uses your CMD as-is
docker run -d -p 8080:5000 myapp:v1
# → runs: gunicorn --workers=2 --bind=0.0.0.0:5000 src.app:app ✓

# Override CMD with a shell (common debugging move)
docker run -it myapp:v1 sh
# → CMD replaced with "sh" → drops you into a shell
# this works because you used CMD, not ENTRYPOINT

# Try to run python instead
docker run -it myapp:v1 python --version
# → CMD replaced with "python --version" → prints Python version

# If you had used ENTRYPOINT ["gunicorn"] instead, the above wouldn't work
# because ENTRYPOINT would always prefix everything with "gunicorn"
```

Your CMD choice means your image is **flexible** — easy to debug, easy to override. That's why for app images, CMD is the right call.

Idea #8 — Hands-on experiment (do this NOW)

This experiment locks the topic in your brain forever. Three minutes of typing.

Experiment 1 — CMD only, see how easily replaced

```
mkdir cmd-test && cd cmd-test

cat > Dockerfile << 'EOF'
FROM ubuntu:22.04
CMD ["echo", "hello from CMD"]
EOF

docker build -t test-cmd .

docker run --rm test-cmd
# → "hello from CMD" (uses CMD as written)

docker run --rm test-cmd echo "different thing"
# → "different thing" (CMD was REPLACED)

docker run --rm test-cmd ls /
# → file listing (CMD completely gone!)
```

Experiment 2 — ENTRYPOINT only, see args appended

```
cat > Dockerfile << 'EOF'
FROM ubuntu:22.04
ENTRYPOINT ["echo", "hello from ENTRYPOINT"]
EOF

docker build -t test-entry .

docker run --rm test-entry
# → "hello from ENTRYPOINT" (just the entrypoint)

docker run --rm test-entry "extra word"
# → "hello from ENTRYPOINT extra word" (APPENDED!)

docker run --rm test-entry ls /
# → "hello from ENTRYPOINT ls /" (literally appended as args to echo)
```

Experiment 3 — Both combined, the real-world pattern

```
cat > Dockerfile << 'EOF'
FROM ubuntu:22.04
ENTRYPOINT ["echo"]
CMD ["default-message"]
EOF

docker build -t test-both .

docker run --rm test-both
# → "default-message"    (ENTRYPOINT + CMD)

docker run --rm test-both "custom message"
# → "custom message"    (ENTRYPOINT stays, CMD replaced)
```

Experiment 4 — Override ENTRYPOINT (debugging)

```
docker run --rm --entrypoint ls test-both /
# → file listing    (entrypoint replaced with ls – bypassed echo entirely)

# Cleanup
cd .. && rm -rf cmd-test
docker rmi test-cmd test-entry test-both
```

WHEN IT CLICKS

Run those 4 experiments and watch the output match what you predicted. The topic becomes permanent. Run them.

Idea #9 — Common mistakes (avoid these)

Mistake 1 — Using shell form on ENTRYPOINT

```
ENTRYPOINT python app.py          # ■ shell form

docker run myimage --debug
# → --debug is SILENTLY DROPPED — your app runs without it
```

Brutal silent failure. Always use exec form.

Mistake 2 — Expecting CMD to survive partial override

```
ENTRYPOINT ["python"]
CMD ["app.py", "--mode=dev"]

docker run myimage app.py
# → runs: python app.py
# user expected: python app.py --mode=dev
# but CMD was replaced ENTIRELY by "app.py" — --mode=dev was lost
```

CMD is all-or-nothing. You can't override just part of it.

Mistake 3 — Multiple CMD or ENTRYPOINT lines

```
CMD ["python", "app.py"]
CMD ["python", "other.py"]  # ■ first one ignored
```

Only the **last** CMD (or ENTRYPOINT) in your Dockerfile takes effect. Earlier ones are silently overridden.

Idea #10 — Self-check (5 questions)

You should be able to answer all 5 instantly:

Q1. What's the difference between CMD and ENTRYPOINT in one line?

→ CMD is replaceable defaults. ENTRYPOINT is a locked-in command that args get appended to.

Q2. Should I write CMD python app.py or CMD ["python", "app.py"]?

→ Always the JSON array (exec form). Shell form breaks signals and silently drops args.

Q3. I have CMD ["python", "app.py"] and run docker run myimage echo hi — what happens?

→ Runs echo hi. CMD was replaced. Your Python app didn't even start.

Q4. I have ENTRYPOINT ["python"] CMD ["app.py"] and run docker run myimage other.py — what happens?

→ Runs `python other.py`. **ENTRYPOINT** stays, **CMD** replaced.

Q5. When should I use **ENTRYPOINT** instead of just **CMD**?

→ When packaging a CLI tool (like `aws`, `kubectl`) where the program is fixed and the user passes subcommands. For a regular web app, just use **CMD**.

Summary in one paragraph

TOPIC 3 — THE WHOLE THING

CMD sets a default command that runs when the container starts — but if you pass arguments to `docker run`, your **CMD** is silently replaced by them. **ENTRYPOINT** sets a "locked-in" command — arguments to `docker run` get appended to it instead of replacing it. When both are used together, **ENTRYPOINT** is the program and **CMD** is the default arguments to that program. Always use **exec form** (`["python", "app.py"]`, not bare strings) so signals reach your app and arguments behave correctly. For a normal web app, **use only CMD** — it's flexible and easy to override during debugging. Use **ENTRYPOINT + CMD** when building a CLI tool image where the program is fixed and users provide subcommands. The 99% rule: if unsure, use **CMD** only.

Level 2 Mastery Checklist

Before moving to Topic 4 (Multi-stage builds), make sure you can confidently do every item below without referring back to notes. If any feels shaky, spend another day on that topic.

TOPIC 1 — Writing Dockerfiles

- Explain what each instruction does: FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, USER, CMD
- Choose the right Python base image variant (slim vs alpine vs full)
- Write a multi-instruction Dockerfile that builds successfully
- Explain why ADD is avoided in favor of COPY
- Add a non-root USER for security
- Decode every line of a docker build output
- Distinguish between docker build (creates image) and docker run (creates container)

TOPIC 2 — Image Layers & Build Caching

- Explain that each Dockerfile instruction creates one layer
- Explain the cache key formula: parent_layer_id + instruction text + file content
- Predict CACHED vs rebuilt for any change to your Dockerfile or files
- Explain the domino (cascading invalidation) effect
- State the golden rule: rare changes top, frequent changes bottom
- Find /var/lib/docker on your system and identify image/, overlay2/, buildkit/
- Explain how copy-on-write keeps image layers untouched while running containers
- Show that Docker is built on Linux: namespaces, cgroups, OverlayFS

TOPIC 3 — CMD vs ENTRYPOINT

- Recite the override rule: CMD replaced by docker run args, ENTRYPOINT appended
- Always use exec form (JSON array), never shell form (bare string)
- Predict the runtime command for any combo of CMD/ENTRYPOINT and docker run args
- Pick CMD-only for app images, ENTRYPOINT+CMD for tool images
- Override ENTRYPOINT for debugging with --entrypoint sh -it
- Explain why shell form silently drops arguments to ENTRYPOINT

Interview Prep — Polished Answers

If an interviewer asks any of these, here's the structured answer to deliver:

Q: What happens when you run `docker build`?

ANSWER (~90 SEC)

When I run `docker build -t myapp:v1 .`, the Docker CLI parses the command and identifies my build context as the current folder. It packs that folder into a tarball, respecting `.dockerignore`, and ships it over a Unix socket to the Docker daemon. The daemon hands the build off to BuildKit, which parses my Dockerfile into a graph of steps. For each step, BuildKit computes a cache key from the instruction text, the parent layer's hash, and — for COPY/ADD — the SHA of the files being copied. If the cache key matches a previously built layer, that layer is reused. Otherwise, BuildKit spins up a temporary container, runs the instruction inside, captures the filesystem diff as a new immutable layer, and discards the container. After all instructions are processed, BuildKit assembles the final image — a stack of layers plus a manifest JSON listing them and a config JSON holding metadata. The image ID is the SHA256 of the config. My tag `myapp:v1` is attached as a pointer to that SHA. The image is stored under `/var/lib/docker/overlay2`, but no container is running — `docker build` only produces the image.

Q: How does Docker layer caching work?

ANSWER (~60 SEC)

Docker caches every layer it builds, indexed by a cache key. The cache key for an instruction is computed from the parent layer's hash, the exact text of the instruction, and — for COPY/ADD — the SHA256 of the files being copied. When Docker processes a Dockerfile, it computes the cache key for each instruction in order. If the key matches a previously-built layer, Docker reuses that layer entirely. If the key doesn't match, Docker rebuilds that layer and *all subsequent layers* — this is cascading invalidation. The practical implication is that the order of instructions matters enormously. You put rarely-changing operations like installing dependencies at the top, and frequently-changing operations like copying source code at the bottom. The classic Python pattern is to copy `requirements.txt` first, install deps, then copy the source code, so editing code doesn't invalidate the dependency-install layer.

Q: What's the difference between CMD and ENTRYPOINT?

ANSWER (~45 SEC)

CMD provides a default command and arguments that run when the container starts. ENTRYPOINT defines the executable that always runs. The key difference is what happens with command-line arguments: arguments to `docker run` *replace* CMD entirely, but *get appended* to ENTRYPOINT. So CMD is overridable defaults; ENTRYPOINT is locked-in behavior. When both are used, ENTRYPOINT is the program and CMD is the default arguments to it. Always use exec form — the JSON array `["python", "app.py"]` — not the shell form, because shell form forks an extra `/bin/sh` process that breaks signal handling and silently drops command-line arguments to ENTRYPOINT. For a typical web app use CMD only. For a CLI tool image, use ENTRYPOINT plus CMD so users can pass subcommands.

Q: Where does Docker store its data?

ANSWER (~45 SEC)

All Docker data lives under `/var/lib/docker/` on Linux. The `image/` folder is the catalog — it stores manifests and config JSONs that describe which layers belong to which images, plus tag-to-SHA mappings. The `overlay2/` folder is the warehouse — each layer is one folder named by its SHA hash, containing a `diff/` subfolder with the actual files that layer added. The `buildkit/` folder is the build cache database — it stores key-value mappings from cache fingerprints to layer SHAs. When BuildKit hits a CACHED step, it's looking up the cache key in `buildkit/`, getting back a layer SHA, and reusing the files in `overlay2/` without rerunning anything.

What's Next — Topics 4, 5, 6

You've completed the foundational half of Level 2. The remaining topics build directly on what you've learned:

Topic 4 — Multi-stage builds

Separate build environment from runtime environment. Ship 50 MB production images instead of 1.2 GB ones. The secret to professional Dockerfiles.

Topic 5 — .dockerignore

The file you didn't know you needed. Reduces build context size, speeds up builds, keeps secrets out, makes COPY . . safe.

Topic 6 — Image optimization for production

Slim base images, layer reduction, security best practices, image scanning, distroless images. Make your images both small and secure.

FINAL THOUGHT

You have just completed the conceptual core of professional Docker work. Layer caching, instruction order, build internals, CMD vs ENTRYPOINT — these are the things senior engineers know instinctively and junior engineers struggle with for years. You asked the right questions. You pushed for depth. You earned this. **Practice the hands-on exercises until every command feels natural. Then come back for Topics 4, 5, 6.**